Practice Midterm 2: Food Delivery · Answer Key

Every answer below was run in R; lines starting with #> are R's actual output. Simulation results match only with the same set.seed() and code order.

Problem 1: Functions

1.1 (5 pts)

Each order is late with probability `p_late`, independently of the others. Write a function named **orders_until_late** with one argument, `p_late`, that simulates orders one at a time until the first late order and returns how many orders that took. Run it once with `p_late = 0.2` after `set.seed(1)`, then use `replicate()` to estimate the average over 10,000 runs. What value would you expect?

```
orders_until_late <- function(p_late) {
  orders <- 0
  repeat {
    orders <- orders + 1
    if (runif(1) < p_late) break       # TRUE with probability p_late
  }
  orders
}
set.seed(1)
orders_until_late(0.2)
#> [1] 10
mean(replicate(10000, orders_until_late(0.2)))   # theory: 1 / 0.2 = 5
#> [1] 5.0115
```

Answer: One run took **10** orders; the average over 10,000 runs is **5.01**, close to the expected **1/p = 1/0.2 = 5**. `runif(1) < p_late` is TRUE with probability p_late (`sample(c(TRUE, FALSE), 1, prob = c(p_late, 1 - p_late))` works too).

On your site: Core 02 · Roll until something happens; Core 03 · Geometric; Rec 4 Problem 5

1.2 (5 pts)

A delivery app charges a fee based on distance: $2.99 for trips of 3 km or less, $4.99 for more than 3 km up to 7 km, and $6.99 for longer trips. Write a function named **delivery_fee** that takes a vector of distances and returns the fee for each one, without using a loop. Test it with `c(1.5, 3, 3.01, 7, 12)`.

```
delivery_fee <- function(distance) {
  ifelse(distance <= 3, 2.99,
         ifelse(distance <= 7, 4.99, 6.99))
}
delivery_fee(c(1.5, 3, 3.01, 7, 12))
#> [1] 2.99 2.99 4.99 4.99 6.99
```

Answer: `2.99 2.99 4.99 4.99 6.99`. Check the boundaries: exactly 3 km is still $2.99 (<=), 3.01 km is $4.99, and exactly 7 km is $4.99. Nested `ifelse()` handles the whole vector at once.

On your site: Rec 3 Problem 1 Question 3 (nested ifelse); Core 05 · if() on a whole vector

1.3 (5 pts)

Orders arrive in the Morning, Afternoon, Evening or Night with probabilities 0.20, 0.30, 0.35 and 0.15. Write a function named **shift_orders** with one argument, n, that simulates the time slot of n orders and returns a list with (a) a frequency table that shows **all four** time slots in that order, and (b) the name of the busiest time slot. Test it with `set.seed(2)` and `n = 15`.

```
shift_orders <- function(n) {
  slots <- c("Morning", "Afternoon", "Evening", "Night")
  orders <- sample(slots, n, replace = TRUE, prob = c(0.20, 0.30, 0.35, 0.15))
  counts <- table(factor(orders, levels = slots))
  list(counts = counts, busiest = names(which.max(counts)))
}
set.seed(2)
shift_orders(15)
#> $counts
#>   Morning Afternoon   Evening     Night
#>         3         5         5         2
#> $busiest
#> [1] "Afternoon"
```

Answer: Morning 3, Afternoon 5, Evening 5, Night 2. The busiest slot is "**Afternoon**": there is a tie with Evening, and `which.max()` returns the **first** maximum. Mention ties when you see them. `factor(..., levels = slots)` keeps the four slots in the requested order and shows a 0 for any slot that never came up.

On your site: Core 01 · Unfair dice; Core 05 · table() leaves out categories; Core 06 · Which condition tool? (which.max)

1.4 (5 pts)

Write a function named **late_summary** with arguments n, `mean_time`, `sd_time` and `limit` (default 45). It should simulate n delivery times from a normal distribution, and return a named vector with the **proportion of deliveries longer than** `limit` and the **first quartile, median and third quartile** of the simulated times. Test it with `set.seed(3)`, `n = 200`, `mean_time = 38`, `sd_time = 9`. What is the exact probability that one delivery is late?

```
late_summary <- function(n, mean_time, sd_time, limit = 45) {
  times <- rnorm(n, mean = mean_time, sd = sd_time)
  q <- quantile(times, c(0.25, 0.50, 0.75))
  c(prop_late = mean(times > limit), Q1 = q[[1]], Median = q[[2]], Q3 = q[[3]])
}
set.seed(3)
round(late_summary(n = 200, mean_time = 38, sd_time = 9), 3)
#> prop_late        Q1    Median        Q3
#>     0.230    31.465    38.567    44.544
1 - pnorm(45, mean = 38, sd = 9)                # exact probability of a late delivery
#> [1] 0.21835
```

Answer: About **23%** of the 200 simulated deliveries were late; the exact probability is 1 - pnorm(45, 38, 9) = **0.218**. Simulated quartiles: Q1 ≈ 31.5, median ≈ 38.6, Q3 ≈ 44.5 minutes (close to the true median of 38). `mean(times > limit)` gives the proportion late.

On your site: Core 03 · Normal probabilities and rnorm; Rec 3 Problem 1 Question 2 (quantile)

Problem 2: Food delivery data

2.1 (2 pts)

Import `food_delivery.csv` into R, converting all character variables into factors. Display the structure of the data, and report how many values are missing in each column.

```
delivery <- read.csv("food_delivery.csv", stringsAsFactors = TRUE)
str(delivery)
#> 'data.frame':	500 obs. of  9 variables:
#>  $ Order_ID              : int  1 2 3 4 5 6 7 8 9 10 ...
#>  $ Distance_km           : num  9.31 16.35 5.39 8.85 10.41 ...
#>  $ Weather               : Factor w/ 5 levels "Clear","Foggy",..: 2 1 1 4 1 1 5 1 1 5 ...
#>  $ Traffic_Level         : Factor w/ 3 levels "High","Low","Medium": 3 2 3 1 2 3 3 3 1 3 ...
#>  $ Time_of_Day           : Factor w/ 4 levels "Afternoon","Evening",..: 2 3 1 1 4 1 1 2 1 1 ...
#>  $ Vehicle_Type          : Factor w/ 3 levels "Bike","Car","Scooter": 3 3 1 1 2 2 3 3 1 1 ...
#>  $ Preparation_Time_min  : int  28 11 29 15 21 18 9 26 8 29 ...
#>  $ Courier_Experience_yrs: int  4 6 1 7 5 4 0 5 8 5 ...
#>  $ Delivery_Time_min     : int  66 65 61 67 64 90 53 50 65 60 ...
colSums(is.na(delivery))                          # missing values in each column
#>               Order_ID            Distance_km                Weather
#>                      0                      0                     15
#>          Traffic_Level            Time_of_Day           Vehicle_Type
#>                     15                      0                      0
#>   Preparation_Time_min Courier_Experience_yrs      Delivery_Time_min
#>                      0                     15                      0
```

Answer: 500 orders and 9 variables. Weather, Traffic_Level, Time_of_Day and Vehicle_Type are factors (levels in **alphabetical** order until you change them). **Missing values:** 15 each in Weather, Traffic_Level and Courier_Experience_yrs. `colSums(is.na(df))` counts them for every column at once.

On your site: Everyday code · Import data, Missing values (NA)

2.2 (3 pts)

What is the average delivery time of all orders? How does the average delivery time compare across vehicle types? Additionally, among **Evening** orders only, how does the average delivery time compare across vehicle types?

```
mean(delivery$Delivery_Time_min)
#> [1] 58.452
aggregate(Delivery_Time_min ~ Vehicle_Type, data = delivery, FUN = mean)
#>   Vehicle_Type Delivery_Time_min
#> 1         Bike          58.82126
#> 2          Car          59.60284
#> 3      Scooter          56.88158
evening <- delivery[delivery$Time_of_Day == "Evening", ]
aggregate(Delivery_Time_min ~ Vehicle_Type, data = evening, FUN = mean)
#>   Vehicle_Type Delivery_Time_min
#> 1         Bike          58.33333
#> 2          Car          61.39130
#> 3      Scooter          55.59184
```

Answer: The average delivery takes **58.45** minutes. Cars are slowest on average (**59.60**), then bikes (**58.82**), then scooters (**56.88**). Among Evening orders the gap widens: cars **61.39**, bikes **58.33**, scooters **55.59**.

On your site: HW 2 Problem 2 Part 1 and its practice problem

2.3 (2 pts)

Which order took the longest to deliver (give its index)? Display that whole row. Conversely, which order was delivered the fastest?

```
which.max(delivery$Delivery_Time_min)
#> [1] 169
delivery[which.max(delivery$Delivery_Time_min), ]
#>     Order_ID Distance_km Weather Traffic_Level Time_of_Day Vehicle_Type
#> 169      169       19.06   Rainy          High   Afternoon         Bike
#>     Preparation_Time_min Courier_Experience_yrs Delivery_Time_min
#> 169                   19                      1               104
which.min(delivery$Delivery_Time_min)
#> [1] 171
delivery[which.min(delivery$Delivery_Time_min), c("Order_ID", "Distance_km", "Delivery_Time_min")]
#>     Order_ID Distance_km Delivery_Time_min
#> 171      171        1.92                10
```

Answer: Order **169** took the longest: **104** minutes (19.06 km, Rainy, High traffic, by bike, a courier with 1 year of experience). Order **171** was the fastest: 10 minutes for 1.92 km. Printing the whole row (`df[i, ]`) explains why it was so slow.

On your site: HW 2 Problem 2 Part 2

2.4 (3 pts)

What is the median delivery time for each traffic level? Answer the same question again, now also taking Weather into account (every combination of traffic level and weather). How many orders are left out of this table, and why?

```
aggregate(Delivery_Time_min ~ Traffic_Level, data = delivery, FUN = median)
#>   Traffic_Level Delivery_Time_min
#> 1          High                65
#> 2           Low                55
#> 3        Medium                60
tapply(delivery$Delivery_Time_min, list(delivery$Traffic_Level, delivery$Weather), median)
#>        Clear Foggy Rainy Snowy Windy
#> High    63.0  80.0  65.5  67.0  65.5
#> Low     52.5  43.5  55.0  62.0  56.0
#> Medium  57.0  66.0  58.5  62.5  61.0
sum(is.na(delivery$Traffic_Level) | is.na(delivery$Weather))   # orders left out
#> [1] 30
```

Answer: Median delivery time rises with traffic: Low **55**, Medium **60**, High **65** minutes. In the traffic × weather table, the slowest combination is High traffic in Fog (80). **30 orders** are left out because their Traffic_Level or Weather is missing. `aggregate()` and `tapply()` silently drop NA groups, so say so in your answer.

On your site: HW 2 Problem 2 Part 3; Core 06 · Which summary tool?

2.5 (2 pts)

What is the average courier experience? (Watch out for missing values.) What is the average courier experience for each vehicle type?

```
mean(delivery$Courier_Experience_yrs)                  # NA because of missing values
#> [1] NA
mean(delivery$Courier_Experience_yrs, na.rm = TRUE)
#> [1] 4.531959
tapply(delivery$Courier_Experience_yrs, delivery$Vehicle_Type, mean, na.rm = TRUE)
#>     Bike      Car  Scooter
#> 4.325000 4.459854 4.878378
```

Answer: Without `na.rm = TRUE` the mean is **NA**, because 15 values are missing. With it, the average courier has **4.53** years of experience (Bike 4.33, Car 4.46, Scooter 4.88). In `tapply()`, extra arguments like `na.rm = TRUE` go after the function.

On your site: Core 05 · NA spreads: use na.rm = TRUE; Everyday code · Missing values

2.6 (4 pts)

Quartiles. Find the first, second and third quartiles of Distance_km. Compare the average delivery time of orders **longer than the third quartile** with that of orders **shorter than the first quartile**. Then add a new factor variable named `Distance_Group` with levels Short (at or below Q1), Medium (above Q1, up to Q3) and Long (above Q3), without modifying Distance_km, and report how many orders fall in each group.

```
q <- quantile(delivery$Distance_km, c(0.25, 0.50, 0.75))
q
#>    25%    50%    75%
#>  5.150  9.680 15.165
mean(delivery$Delivery_Time_min[delivery$Distance_km > q[3]])   # longest quarter of trips
#> [1] 78.976
mean(delivery$Delivery_Time_min[delivery$Distance_km < q[1]])   # shortest quarter of trips
#> [1] 38.19355
delivery$Distance_Group <- cut(delivery$Distance_km, breaks = c(-Inf, q[1], q[3], Inf),
                               labels = c("Short", "Medium", "Long"))
table(delivery$Distance_Group)
#>  Short Medium   Long
#>    126    249    125
```

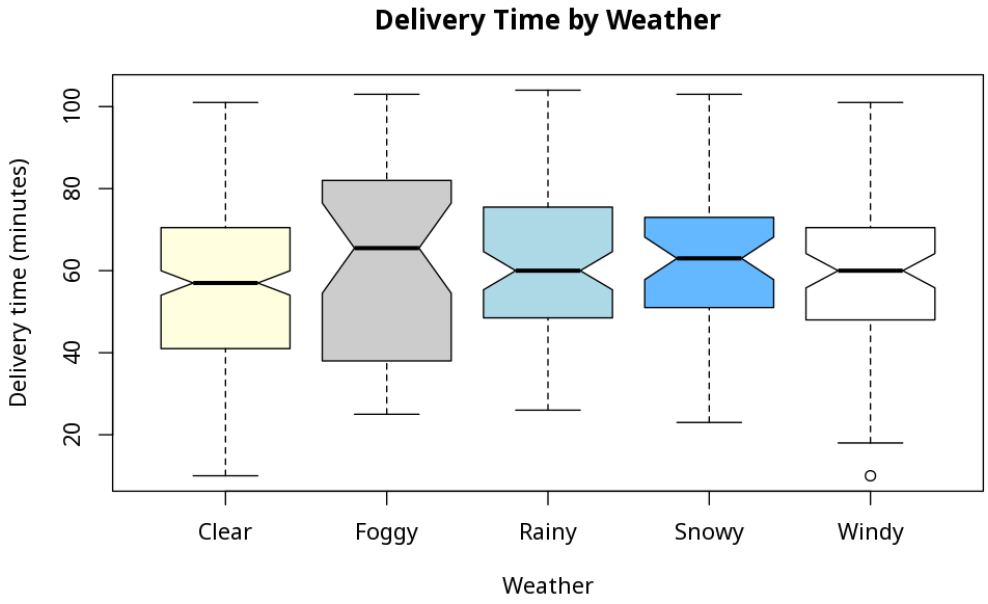
Answer: Quartiles of distance: Q1 = **5.15** km, median = **9.68** km, Q3 = **15.17** km. Orders longer than Q3 average **78.98** minutes, versus **38.19** for orders shorter than Q1, so the longest quarter of trips takes about twice as long. Groups: **Short 126, Medium 249, Long 125**. `cut()` with `breaks = c(-Inf, Q1, Q3, Inf)` includes each right end, matching "at or below Q1".

On your site: Rec 3 Problem 1 Question 2 (percentiles); Rec 3 Question 3 and Lecture 3 (cut)

2.7 (3 pts)

Compare delivery times across weather conditions using boxplots. Customize the boxplots by adding labels, changing the colors and adding notches. Are there any outliers?

```
b <- boxplot(Delivery_Time_min ~ Weather, data = delivery,
             notch = TRUE,
             col = c("lightyellow", "grey80", "lightblue", "steelblue1", "white"),
             main = "Delivery Time by Weather",
             xlab = "Weather", ylab = "Delivery time (minutes)")
b$out
#> [1] 10
```



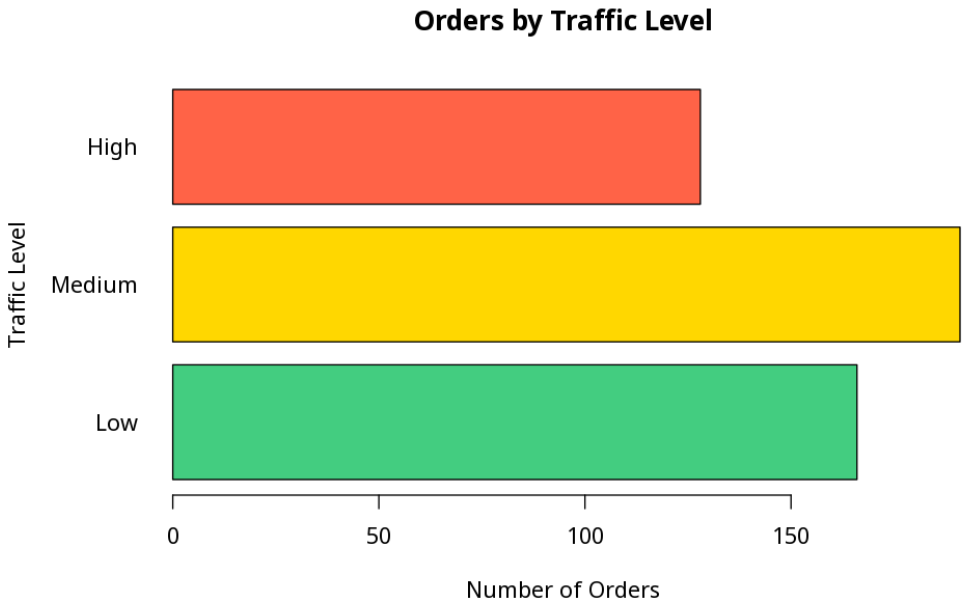
Answer: One outlier: a **10**-minute Windy delivery (order 171, a short trip). Clear weather has the lowest median (57) and Foggy the highest (65.5); Rainy, Snowy and Windy are around 60 to 63. The Foggy box is the widest (only 40 orders), so its notch is wide too. The 15 orders with missing Weather are not plotted.

On your site: HW 2 Problem 2 Part 4; Core 04 · Boxplot by group

2.8 (3 pts)

Provide an overview of traffic levels using a barplot. Display the bars in the order Low, Medium, High, make them horizontal, and customize the plot with labels and colors. Are the missing traffic values shown?

```
delivery$Traffic_Level <- factor(delivery$Traffic_Level, levels = c("Low", "Medium", "High"))
par(mar = c(5, 7, 4, 2))
barplot(table(delivery$Traffic_Level),
        horiz = TRUE, las = 1,
        col = c("seagreen3", "gold", "tomato"),
        xlab = "Number of Orders",
        main = "Orders by Traffic Level")
title(ylab = "Traffic Level", line = 5)
par(mar = c(5, 4, 4, 2))
table(delivery$Traffic_Level, useNA = "ifany")          # NAs are not drawn
#>    Low Medium   High   <NA>
#>    166    191    128     15
```



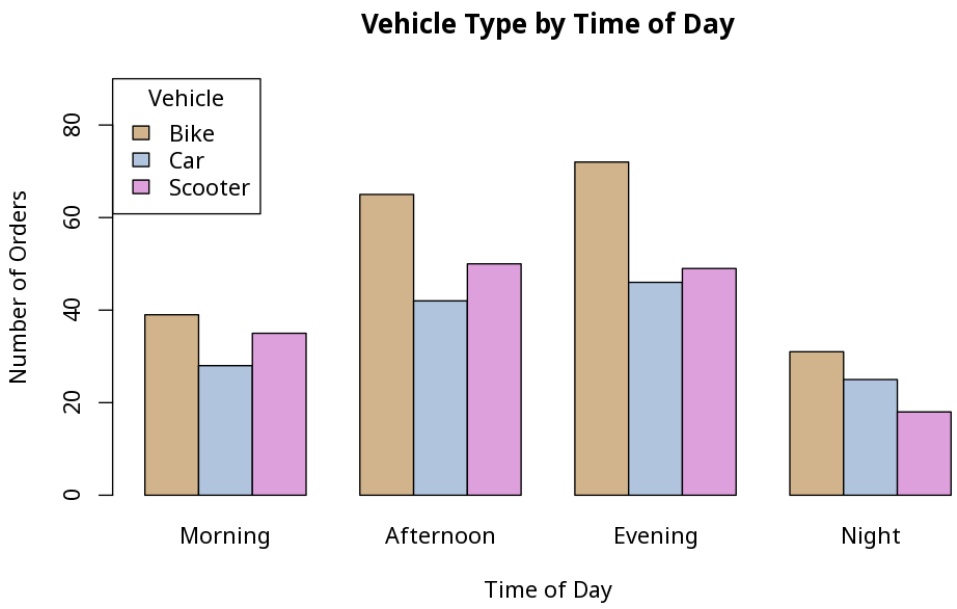
Answer: Medium traffic is the most common (191 orders), then Low (166) and High (128). The **15 missing values are not drawn**; `table(x, useNA = "ifany")` shows them. Reorder the levels first so the bars follow Low, Medium, High (the first level is drawn at the bottom), and widen the left margin so the names fit.

On your site: HW 2 Problem 2 Part 5; Core 04 · Barplot

2.9 (3 pts)

Create side-by-side barplots showing the number of orders of each vehicle type for each time of day (in the order Morning, Afternoon, Evening, Night). Include a legend, and customize the plot as in the previous question.

```
delivery$Time_of_Day <- factor(delivery$Time_of_Day,
                               levels = c("Morning", "Afternoon", "Evening", "Night"))
barplot(table(delivery$Vehicle_Type, delivery$Time_of_Day),
        beside = TRUE,
        col = c("tan", "lightsteelblue", "plum"),
        legend.text = TRUE,
        args.legend = list(x = "topleft", title = "Vehicle"),
        ylim = c(0, 90),
        xlab = "Time of Day", ylab = "Number of Orders",
        main = "Vehicle Type by Time of Day")
```



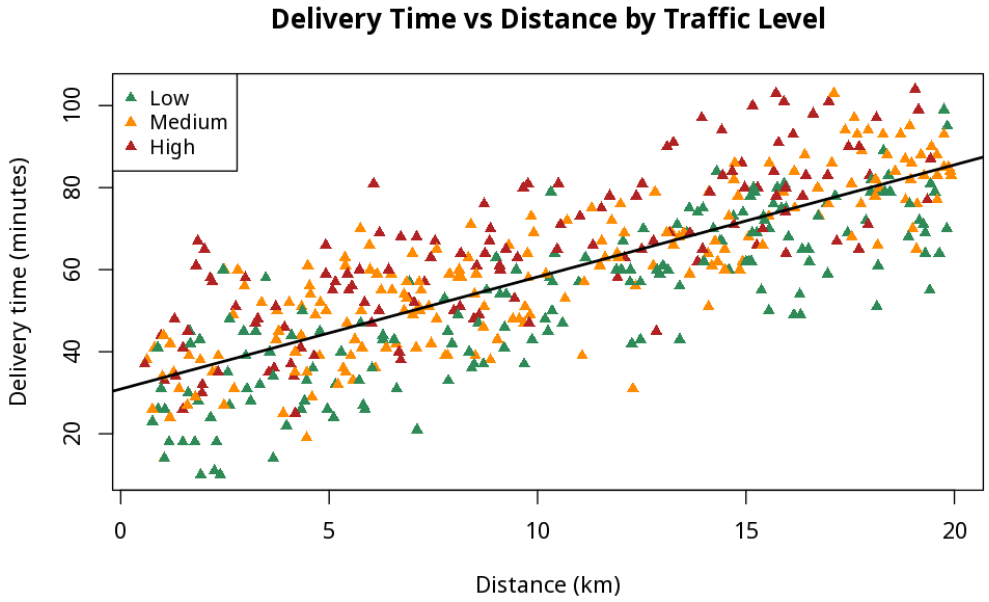
Answer: Bikes are the most used vehicle at every time of day, and the Evening is the busiest time (72 bike, 46 car, 49 scooter orders). Night has the fewest orders. Setting the Time_of_Day levels puts the groups in clock order instead of alphabetical.

On your site: HW 2 Problem 2 Part 6; Core 04 · Side-by-side barplot

2.10 (3 pts)

Plot delivery time against distance. Label the plot and axes properly. Use triangular points of size 0.9, colored by traffic level (three colors), add a legend and a regression line. Describe the relationship you see.

```
traffic_cols <- c(Low = "seagreen", Medium = "darkorange", High = "firebrick")
plot(delivery$Distance_km, delivery$Delivery_Time_min,
     pch = 17, cex = 0.9,
     col = traffic_cols[as.character(delivery$Traffic_Level)],   # NA traffic -> not drawn
     xlab = "Distance (km)", ylab = "Delivery time (minutes)",
     main = "Delivery Time vs Distance by Traffic Level")
abline(lm(Delivery_Time_min ~ Distance_km, data = delivery), lwd = 2)
legend("topleft", legend = names(traffic_cols), col = traffic_cols, pch = 17, cex = 0.9)
cor(delivery$Distance_km, delivery$Delivery_Time_min)
#> [1] 0.7976197
```



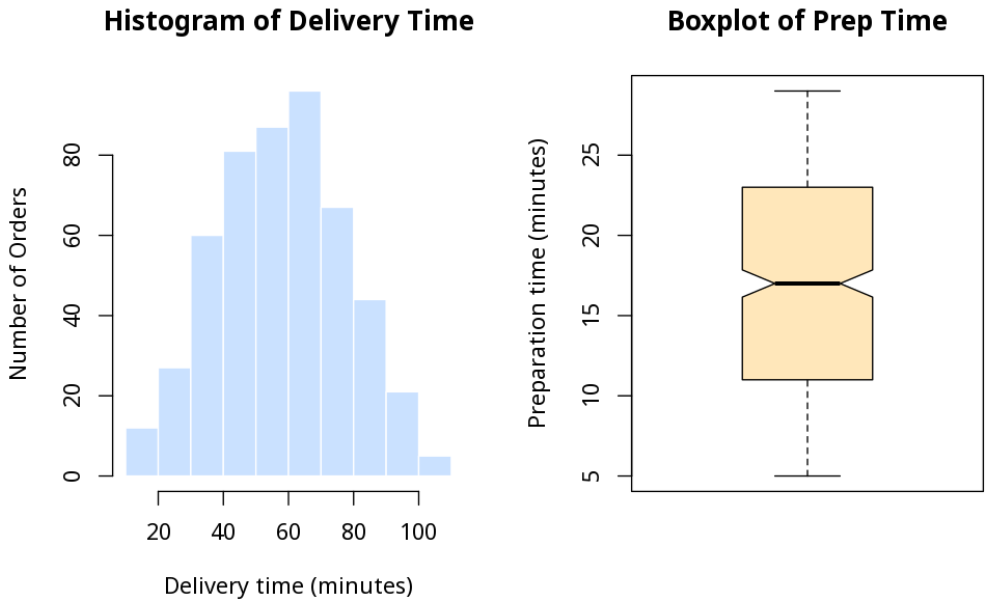
Answer: A **strong positive** relationship (correlation **0.80**): longer trips take longer, about 2.7 more minutes per km (the slope of the regression line). At the same distance, High-traffic orders (red) sit above Low-traffic ones (green). Orders with missing traffic get no color and are not drawn; the regression line uses all orders.

On your site: HW 2 Problem 2 Part 7; Core 04 · Scatterplot with a trend line

2.11 (2 pts)

Combine a histogram of delivery time (about 10 bins) and a boxplot of preparation time into a single image. Customize both plots as in Question 2.7 (no notch on the histogram).

```
par(mfrow = c(1, 2))
hist(delivery$Delivery_Time_min, breaks = 10,
     col = "lightsteelblue1", border = "white",
     xlab = "Delivery time (minutes)", ylab = "Number of Orders",
     main = "Histogram of Delivery Time")
boxplot(delivery$Preparation_Time_min, col = "wheat1", notch = TRUE,
        ylab = "Preparation time (minutes)", main = "Boxplot of Prep Time")
par(mfrow = c(1, 1))
```



Answer: Delivery time is roughly bell-shaped, centered near 60 minutes (median 60), ranging from 10 to 104. Preparation time is spread evenly between 5 and 29 minutes (median 17) with no outliers.

On your site: HW 2 Problem 2 Part 8; Core 04 · Several plots in one window